

포인터 접근의 해결 방식

-기존 방식의 문제-

첫째, 파일이 크면 다시 통째로 꺼내야 해서 비효율적이다.

둘째, LLM이 "145~190번째 줄만 보고 싶다" 같은 정밀 접근을 못 한다.

셋째, 한 함수만 수정하면 되는데 파일 전체를 계속 다뤄야 한다.

넷째, 코드 수정 시 문맥 범위를 너무 넓게 잡아 불필요한 토큰이 늘어난다.

즉, **pointer** 방식의 본질적 한계라기보다, 포인터 설정 범위를 너무 거칠게 잡았을 때 생기는 구현상의 한계라고 보는 게 맞다.

-외부 파일 포인터

1. runtime pointer

실행 중 생성된 외부 객체에 대한 참조자

2. source pointer

디스크/외부 저장소의 파일 객체에 대한 참조자

코드가 아닌 외부 데이터는 종류가 너무 다양하다.

예를 들면

- 긴 로그 파일

- PDF 문서
- 마크다운 문서
- JSON
- CSV
- DB 결과
- HTML
- 내부 정책 문서
- 사양서

이런 것들을 전부 하나의 통일된 하위 포인터 체계로 다루려고 하면 금방 과설계가 된다.

예를 들어

- 로그에는 line-span이 의미 있을 수도 있지만
- PDF에는 line-span이 애매하고
- JSON에는 line-span보다 key path가 더 자연스럽고
- CSV는 row/column이 더 맞고
- DB는 SQL query가 더 맞다

즉 외부 데이터는 **저장 위치보다 데이터 형식이 더 중요하다.**

그래서 이런 경우엔 “하위 포인터를 설계한다”보다

“해당 형식에 맞는 tool을 만든다”가 더 자연스럽다.

-내부 코드 포인터

1. line-span selector

파일 내부 줄 범위 선택자

2. **symbol selector**

파일 내부 심볼 선택자

3. **symbol index / file outline**

selector를 resolve하기 위한 메타데이터

1단계

“실제 수정 및 타겟 지정”을 위한 **line-span selector** 지원

memory://path#Lx-Ly

2단계

file outline / symbol index 추가

각 파일에 대해 함수명, 클래스명, 시작줄/끝줄을 json 형태로 parser를 통해 정리

3단계

symbol selector 지원

memory://symbol/file.py::func_name

위의 3단계 operation을 통해 코드의 구조 및 각 파일 별 클래스, 함수, 메소드 목록이 나오고, 이는 json파일로 저장되어 프로젝트 하위 폴더에 기록 된다. 추후 이 json은 외부 파일 포인터를 통해 tool을 이용하여 참조된다.

저희는 파일 전체의 포인터 개념을 줄 범위와 함수/클래스 단위까지 확장할 계획입니다. 즉 runtime memory에서 file-level pointer뿐만 아니라 span-level pointer와 symbol-level pointer를 함께 관리하여, LLM이 필요한 코드 일부만 선택적으로 읽고 수정할 수 있도록 설계하겠습니다."